

1. Captura de datos

➤ Ejemplo (metáfora)

La **captura de datos** es como el **sistema de tuberías de una ciudad**:

- El agua son los datos.
- Las tuberías son las tecnologías que transportan esos datos.
- El depósito final es el Data Lake o Data Warehouse, donde se almacenan para analizarlos.

No importa de dónde venga el agua (lluvia, río, pozo...), lo importante es llevarla al mismo sitio.

➤ Objetivos

"El objetivo es **recoger datos desde muchas fuentes distintas y llevarlos a un sistema central, en tiempo real o en bloques (por lotes).**"

Fuentes típicas:

- **APIs**: como cuando una app pide datos del tiempo o Google Maps.
- **Sensores/IoT**: un smartwatch que mide pasos o un sensor de temperatura.
- **Logs**: registros de actividad de una app o servidor.
- **Bases de datos**: información de clientes en un sistema tradicional.
- **Redes sociales / clics**: cada "like" o cada clic en una web.

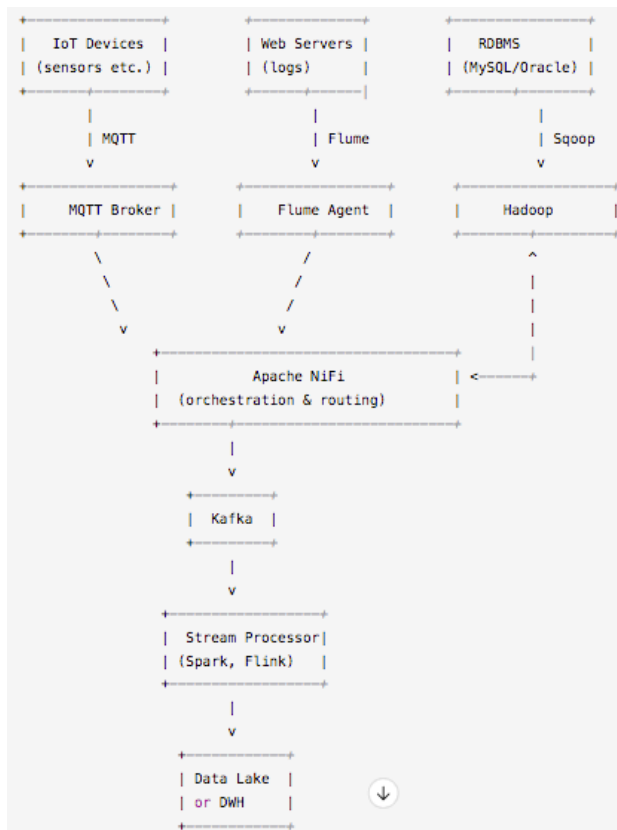
➤ Tecnologías con ejemplos

- **MQTT** "El WhatsApp de las máquinas": sensores envían mensajes pequeñitos a un broker.
- **Flume** "El mensajero de los logs": recoge registros de servidores y los manda a sistemas grandes como Hadoop.
- **Sqoop** "El puente" entre bases de datos tradicionales (MySQL, Oracle...) y Hadoop.
- **NiFi** "El director de orquesta": recibe datos de varias fuentes y decide adónde enviarlos.
- **Kafka** "Un buzón gigante en tiempo real": guarda mensajes (eventos) para que otros sistemas (ej: Spark) los procesen.

➤ Flujo visual

Esquema:

- Fuentes de datos → (APIs, sensores, logs, redes sociales, BBDD)
- Tecnologías de ingesta (MQTT, Flume, Sqoop) →
- Orquestador (NiFi) →
- Cola de eventos (Kafka) →
- Procesamiento (Spark, etc.) →
- Destino final (Data Lake, DWH).



➤ Ejemplo práctico

“Imaginad que tenéis una app de fitness. Cada vez que corréis:

- El sensor del móvil mide pasos (IoT → MQTT).
- La app genera registros de uso (logs → Flume).
- La API de Google Maps da la ruta (API REST).
- Todo eso se manda a NiFi, luego a Kafka, y finalmente acaba en un Data Lake donde podemos analizar: ¿qué ciudad corre más?, ¿qué día de la semana?, ¿a qué velocidad?”.

2. Almacenamiento Distribuido

➤ Ejemplo (metáfora)

Almacenar datos en informática es como guardar libros en una **biblioteca**:

- Una sola estantería (servidor) se llena rápido.
- Si quieres escalar, necesitas **varias estanterías repartidas** (almacenamiento distribuido).
- Cada estantería tiene una **copia de seguridad** de libros importantes → eso es la **tolerancia a fallos**.

Por eso el “almacenamiento distribuido” significa **repartir datos en varios nodos/máquinas para que no se pierdan y sea más fácil crecer**.

➤ Objetivo principal

“El objetivo es **guardar enormes volúmenes de datos de todo tipo** (texto, vídeos, sensores...) de forma segura, escalable y sin que se pierda nada aunque un servidor falle.”

➤ Características clave

- **Soporta todo tipo de datos:**
 - **Estructurados** → tablas SQL.
 - **Semiestructurados** → JSON, XML, logs.
 - **No estructurados** → fotos, vídeos, audios.
- **Escalabilidad horizontal:** si se queda corto, no amplias el “servidor para que sea más grande”, sino que **añades más nodos/máquinas**.
- **Tolerancia a fallos:** los datos se replican, si un nodo cae, otro tiene la copia.

➤ Tecnologías comunes (con ejemplos simples)

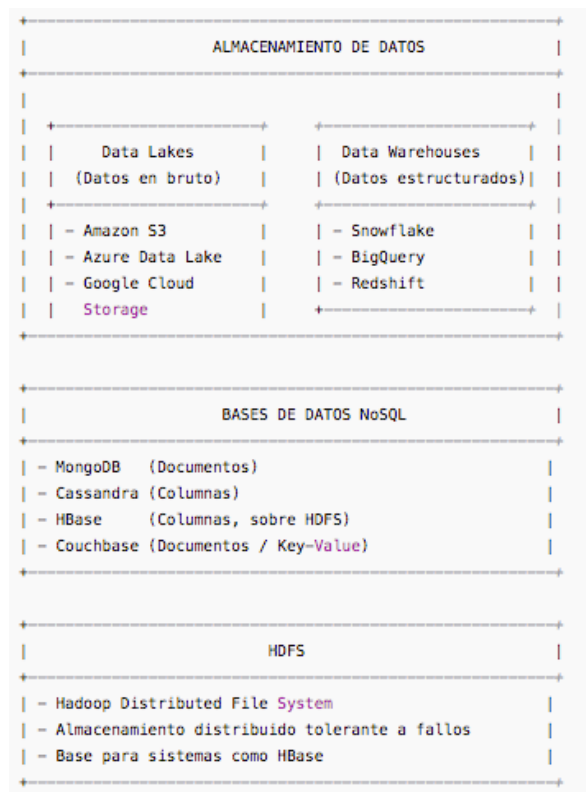
- **Data Lake**
 - “Un lago donde echas todo el agua sin filtrar.”
Se guardan los datos en bruto, sin transformar, en cualquier formato. Ej: logs, imágenes, CSV, JSON.
- **Data Warehouse (DWH)**
 - “Una biblioteca ordenada y limpia.”
Solo se guardan datos estructurados, listos para análisis con SQL. Ej: informes de ventas, KPIs.
- **NoSQL**
 - “Cajas flexibles según lo que quieras guardar.”
Bases de datos que no usan tablas clásicas, adaptadas a documentos, grafos, columnas o claves-valor. Ej: MongoDB, Cassandra.
- **HDFS (Hadoop Distributed File System)**
 - “El clásico almacén de Big Data.”
Reparte ficheros en trozos por varios nodos, con copias redundantes para que nunca se pierdan.

➤ Ejemplo

“Imaginad que Netflix necesita guardar:

- La ficha de cada película (estructurado → Data Warehouse).
- Las reseñas y comentarios (semiestructurado → NoSQL).
- Los propios vídeos en HD (no estructurado → Data Lake).

Todo eso se reparte en cientos de servidores con HDFS para que si uno falla, no se pierda nada.”



3. Procesamiento masivo

➤ Ejemplo (metáfora)

El **procesamiento de datos** es como **cocinar en una cocina industrial**:

- Recibes toneladas de ingredientes (datos).
- Tienes que **transformarlos** en platos listos (informes, estadísticas, predicciones).
- Puedes cocinar **todo de golpe (batch)** o ir sirviendo **en tiempo real (streaming)**.

➤ Objetivo

“El objetivo es **transformar y analizar grandes cantidades de datos**, a veces en tiempo real y a veces en bloques más grandes.”

➤ Tipos de procesamiento

- **Batch (por lotes)**

Como preparar la comida del comedor escolar: cocinas todo junto y lo sirves después.

- Alta latencia (los resultados tardan en llegar).
- Mucha complejidad pero útil para análisis globales (ej: procesar todas las ventas de un mes).
- Tecnologías: **Hadoop MapReduce, Apache Spark.**

- **Streaming (tiempo real)**

Como un bar de tapas: cocinas y sirves al momento, plato a plato.

- Baja latencia (respuestas casi inmediatas).
- Ideal para detectar fraudes, analizar clics en una web o datos de sensores IoT.
- Tecnologías: **Apache Flink, Apache Storm, Apache Samza, Spark Structured Streaming.**

➤ Tecnologías unificadas

Hoy en día hay frameworks que hacen **ambas cosas**: batch y streaming.

- Ejemplos: **Apache Beam, Apache Spark, Apache Flink.**
Es como un restaurante híbrido: puede cocinar **grandes banquetes** y también **tapas al momento**.

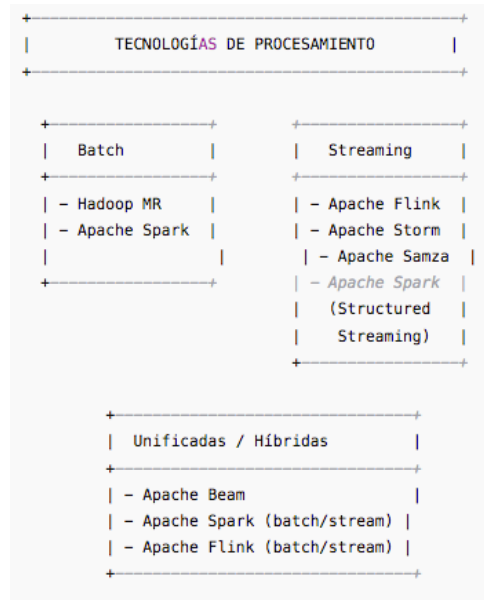
➤ Ejemplo práctico

- **Batch**: Amazon procesa cada noche todas las compras del día para calcular qué productos recomendar.
- **Streaming**: Netflix detecta en tiempo real si demasiados usuarios se desconectan en un servidor (para redirigir el tráfico).
- **Unificado**: Google usa Apache Beam para soportar tanto informes históricos como análisis en tiempo real de búsquedas.

➤ Esquema

Fuentes de datos

- [Batch: Hadoop/Spark] → Informes históricos
- [Streaming: Flink/Storm] → Alertas en tiempo real
- [Unificado: Beam/Spark/Flink] → Ambos



4. Análisis y Visualización

➤ Ejemplo (metáfora)

Todo el proceso de Big Data es como **cultivar un huerto**:

- Captura = sembrar semillas (datos).
- Almacenamiento = guardar los productos en un almacén (Data Lake/DWH).
- Procesamiento = clasificar, limpiar, preparar.
- **Análisis y visualización = cocinar y presentar un plato bonito que todos puedan entender.**

Es decir, convertir datos en **decisiones útiles**.

➤ Objetivo

“El objetivo es **generar información valiosa (insights), modelos predictivos y reportes visuales** que ayuden a tomar decisiones.”

➤ Herramientas y tecnologías (con ejemplos sencillos)

- **Lenguajes de análisis**
 - Python (con Pandas, NumPy, scikit-learn).
 - R (estadística avanzada).
Ejemplo: analizar ventas históricas para ver tendencias.
- **BI & Dashboards**
 - Power BI, Tableau, Looker.
Ejemplo: panel visual con gráficos de ventas por región en tiempo real.
- **Notebooks interactivos**
 - Jupyter, Zeppelin.
Ejemplo: probar un modelo de predicción del clima con datos meteorológicos.
- **ML/IA (Machine Learning / Inteligencia Artificial)**
 - TensorFlow, PyTorch, scikit-learn, MLlib (Spark).
Ejemplo: un modelo que predice si un cliente va a abandonar un servicio (churn prediction).



➤ Ejemplo Completo

“Imaginad que sois Spotify:

- Recogéis datos de cada canción que escuchan los usuarios (captura).
- Los guardáis en servidores distribuidos (almacenamiento).
- Procesáis en tiempo real lo que suena en el mundo (procesamiento).
- **Con el análisis y visualización podéis:**
 - Mostrar dashboards a los managers (ej: la canción más escuchada hoy).
 - Crear recomendaciones personalizadas (IA/ML).
 - Hacer informes visuales para las discográficas.”

➤ Esquema para pizarra

Datos procesados → [Lenguajes de análisis: Python, R]
 → [Dashboards BI: Power BI, Tableau]
 → [Notebooks: Jupyter]
 → [ML/IA: TensorFlow, PyTorch]
 → Resultado final: insights, predicciones, gráficos claros

Con esto, tenemos el **ciclo completo del Big Data**:

Captura →
Almacenamiento →
Procesamiento →
Análisis & Visualización.